

Student: Jakub Kwaśniewski

Numer albumu 32430

Sprawozdanie: Problem hetmanów

Metody Programowania

Uczelnia: Politechnika Morska w Szczecinie

Data: 24.05.2026

Wstęp do zadania:

Celem ćwiczenia jest praktyczne zapoznanie się z implementacją tablicową dwóch liniowych struktur danych, stosu oraz kolejki. Stos działa według zasady LIFO co oznacza, że element dodany jako ostatni jest usuwany jako pierwszy natomiast kolejka opiera się na zasadzie FIFO gdzie pierwszy element dodany jest pierwszym obsługiwany. Zadania obejmują analizę gotowych fragmentów kodu w języku C++ ich samodzielne przepisanie oraz rozbudowę o dodatkowe funkcjonalności.

1. Przepisz powyższe fragmenty kodu. W funkcji main() utwórz nowy stos i wykonaj testy każdej metody [isEmpty, isFull, push, pop, peek]. Następnie opisz działanie każdej metody linijka po linijce.

```
int main()
{
    setlocale(LC_CTYPE, "Polish");

    Stack mojStos;
    cout << "Czy stos jest pusty? (1 = Tak, 0 = Nie): " << mojStos.isEmpty() << "\n";

    mojStos.push(10);
    mojStos.push(20);
    mojStos.push(30);

    cout << "Element na samej górze stosu: " << mojStos.peek() << "\n";
    cout << "Czy stos jest pełny?: " << mojStos.isFull() << "\n";
    mojStos.pop();
    cout << "Teraz na górze jest: " << mojStos.peek() << "\n";

    return 0;
}
```

```
Czy stos jest pusty? (1 = Tak, 0 = Nie): 1
Element na samej górze stosu: 30
Czy stos jest pełny?: 0
Teraz na górze jest: 20
```

```
bool isEmpty() {
    return top == -1;
}
```

bool isEmpty() - nagłówek funkcji zwracającej wartość prawda/fałsz

return top == -1;- funkcja sprawdza, czy zmienna top ma wartość -1, Jeśli tak, zwraca true, w przeciwnym razie zwraca false.

```
bool isFull() {
    return top == MAX - 1;
}
```

bool isFull() - nagłówek funkcji zwracającej wartość prawda/fałsz.

return top == MAX - 1;- sprawdza, czy index top osiągnął maksymalny dostępny indeks tablicy, Jeśli tak zwraca true.

```
void push(int x) {
    if (isFull()) {
        cout << "Stack is Full.\n";
        return;
    }
    arr[++top] = x;
}
```

void push(int x) - nagłówek funkcji przyjmującej liczbę x.

if (isFull())- warunek sprawdzający czy stos nie jest już przypadkiem pełny.

cout << "Stack is full.\n"; - jeśli stos jest pełny wypisuje komunikat o błędzie.

return; - przerywa działanie funkcji, aby nie dodać elementu poza tablicę.

arr[++top] = x; - najpierw zwiększa wartość zmiennej top o 1, a następnie pod ten nowy indeks w tablicy arr wpisuje liczbę x.

```
void pop() {
    if (isEmpty()) {
        cout << "Stack EmptyInfo << "\n";
        return;
    }
    top--;
}
```

void pop() - nagłówek funkcji usuwającej element, która nic nie zwraca.

if (isEmpty()) - warunek sprawdzający, czy na stosie w ogóle coś jest.

cout << stackEmptyInfo << "\n"; - jeśli stos jest pusty, wypisuje komunikat informacyjny.

return; - przerywa działanie funkcji, bo nie ma czego usuwać.

top--; - zmniejsza wartość zmiennej top o 1.

```
int peek() {
    if (!isEmpty())
        return arr[top];
    cout << stackEmptyInfo << "\n";
    return -1;
}
```

int peek() - nagłówek funkcji, która zwraca liczbę całkowitą

if (!isEmpty()) - warunek sprawdzający, jeśli stos nie jest pusty.

return arr[top]; - wraca wartość z tablicy arr, która znajduje się pod aktualnym indeksem top.

cout << stackEmptyInfo << "\n"; - Wypisuje komunikat o pustym stosie.

return -1; - zwraca -1 jako sygnał informujący o błędzie

2. W metodzie push znajduje się linijka: arr[++top] = x; Sprawdź, czy działający będzie wariant: arr[top++] = x; Zapisz odpowiedź z uzasadnieniem.

Wariant arr[top++] = x; nie będzie działał poprawnie, ponieważ zastosowanie postinkrementacji powoduje, że program najpierw próbuje wpisać wartość pod aktualny indeks zmiennej top, a dopiero po wykonaniu tej operacji zwiększa jej wartość o jeden.

3. Napisz pętlę, która wygeneruje stos o liczbie elementów podanej przez użytkownika z pseudolosowymi liczbami jako wartościami elementów stosu.

```
int liczbaElementow;
cout << "Podaj liczbę elementów" << endl;
cin >> liczbaElementow;

for (int i = 0; i < liczbaElementow; i++) {
    int losowaLiczba = rand() % 100;
    mojStos.push(losowaLiczba);
    cout << losowaLiczba << " ";
}

return 0;
```

```
Podaj liczbę elementów
3
41 67 34
```

4. Napisz metodę, która zwróci liczbę elementów odłożonych na stosie.

```
int getSize() {
    return top + 1;
}
```

```
cout << "Liczba elementów w stosie wynosi: " << mojStos.getSize() << endl;
```

```
Liczba elementów w stosie wynosi: 6
```

5. Porównaj, czy liczba podana przez użytkownika [punkt 3.] dotycząca liczby elementów w stosie zgadza się z wynikiem metody z punktu 4.

```
if (liczbaElementow == mojStos.getSize()) {
    cout << "Wyniki się zgadzają " << liczbaElementow << " liczba elementów w stosie: " << mojStos.getSize() << endl;
}
else {
    cout << "Liczby się nie zgadzają" << endl;
}
```

```
Liczba elementów w stosie wynosi: 7
Liczby się nie zgadzają
```

6. Napisz metodę o nazwie printStack, która wyświetli po kolei, od góry do dołu wartości wszystkich elementów odłożonych na stosie.

```
void printStack() {
    if (isEmpty()) {
        cout << stackEmptyInfo << "\n";
        return;
    }

    for (int i = top; i >= 0; i--) {
        cout << arr[i] << "\n";
    }
}
```

```
67
41
20
10
```

7. Napisz metodę o nazwie resetStack, która zresetuje stos, czyli przywróci go do stanu początkowego. Następnie wyświetl cały stos metodą printStack, by

udowodnić wykonanie tego podpunktu. Następnie dodaj do stosu i usuń z niego kilka elementów.

```
void resetStack() {
    top = -1;
}

mojStos.resetStack();
mojStos.printStack();
mojStos.push(50);
mojStos.push(60);
mojStos.push(70);
mojStos.printStack();
mojStos.pop();
mojStos.printStack();|

Stack is empty.
70
60
50
60
50
```

8. Przepisz powyższe fragmenty kodu. W funkcji main() utwórz nową kolejkę i wykonaj testy każdej metody [isEmpty, isFull, enqueue, dequeue, peek]. Następnie opisz działanie każdej metody linijka po linijce.

```
int main()
{
    setlocale(LC_CTYPE, "Polish");

    Queue mojaKolejka;
    cout << "Czy kolejka jest pusta? (1 = Tak, 0 = Nie): " << mojaKolejka.isEmpty() << "\n";

    mojaKolejka.enqueue(10);
    mojaKolejka.enqueue(20);
    mojaKolejka.enqueue(30);

    cout << "Pierwszy element w kolejce to: " << mojaKolejka.peek() << "\n";
    cout << "Czy kolejka jest pełna?: " << mojaKolejka.isFull() << "\n";
    mojaKolejka.dequeue();
    cout << "Teraz pierwszy element w kolejce to: " << mojaKolejka.peek() << "\n";

    return 0;
}
```

```
Czy kolejka jest pusta? (1 = Tak, 0 = Nie): 1
Pierwszy element w kolejce to: 10
Czy kolejka jest pełna?: 0
Teraz pierwszy element w kolejce to: 20
```

```
bool isEmpty() {
    return first == last;
}
```

bool isEmpty() - nagłówek funkcji zwracającej wartość prawda/fałsz.

return first == last; - sprawdza, czy indeks początku i końca są sobie równe, jeśli tak, oznacza to, że w kolejce nie ma żadnych elementów i funkcja zwraca true.

```
bool isFull() {
    return last == MAX;
}
```

bool isFull() - nagłówek funkcji zwracającej wartość prawda/fałsz.

return last == MAX; - sprawdza, czy indeks końca osiągnął maksymalny rozmiar tablicy, jeśli tak, zwraca true co oznacza brak wolnego miejsca.

```
void enqueue(int x) {
    if (isFull()) {
        cout << "Queue is full.\n";
        return;
    }
    arr[last++] = x;
}
```

void enqueue(int x) - nagłówek funkcji, która przyjmuje liczbę x.

if (isFull()) - warunek sprawdzający czy kolejka jest już pełna.

cout << "Queue is full.\n" - jeśli brakuje miejsca wyświetla komunikat o błędzie.

return; - przerywa działanie funkcji zapobiegając zapisowi poza tablicę.

arr[last++] = x; - zapisuje liczbę x pod aktualnym indeksem last a następnie zwiększa wartość zmiennej last o jeden.

```
void dequeue() {
    if (isEmpty()) {
        cout << queueEmptyInfo << "\n";
        return;
    }
    first++;
}
```

void dequeue() - nagłówek funkcji, która usuwa element.

if (isEmpty()) - warunek sprawdzający czy kolejka jest pusta.

cout << queueEmptyInfo << "\n"; - jeśli w kolejce nic nie ma wyświetla komunikat o błędzie.

return; - przerywa działanie funkcji, bo nie ma czego usuwać.

first++; - zwiększa indeks first o jeden.

```
int peek() {  
    if (!isEmpty())  
        return arr[first];  
    cout << queueEmptyInfo << "\n";  
    return -1;  
}
```

int peek() - nagłówek funkcji zwracającej liczbę całkowitą

if (!isEmpty()) - warunek sprawdzający, jeśli kolejka nie jest pusta.

return arr[first]; - jeśli warunek jest spełniony funkcja zwraca wartość elementu znajdującego się na samym początku kolejki.

cout << queueEmptyInfo << "\n"; - instrukcja wykona się tylko gdy kolejka była pusta, wypisuje komunikat o braku elementów.

return -1; - zwraca -1 jako informację o błędzie.

9. Napisz pętlę, która wygeneruje kolejkę o liczbie elementów podanej przez użytkownika z pseudolosowymi liczbami jako wartościami elementów kolejki.

```
int liczbaElementow;  
cout << "Podaj liczbę elementów do dodania do kolejki: ";  
cin >> liczbaElementow;  
  
for (int i = 0; i < liczbaElementow; i++) {  
    int losowaLiczba = rand() % 100;  
    mojaKolejka.enqueue(losowaLiczba);  
    cout << losowaLiczba << " " << endl;  
}
```

```
Podaj liczbę elementów do dodania do kolejki: 3  
41  
67  
34
```

10. Napisz metodę, która zwróci liczbę elementów znajdujących się w kolejce.

```
int getSize() {  
    return last - first;  
}
```

```
cout << "Liczba elementów w kolejce wynosi: " << mojaKolejka.getSize() << "\n";
```

```
Liczba elementów w kolejce wynosi: 7
```

11. Porównaj, czy liczba podana przez użytkownika [punkt 8.] dotycząca liczby elementów w kolejce zgadza się z wynikiem metody z punktu 9.

```
if (liczbaElementow == mojaKolejka.getSize()) {  
    cout << "Wyniki się zgadzają " << liczbaElementow << "liczba elementów w kolejce: " << mojaKolejka.getSize() << "\n";  
}  
else {  
    cout << "Liczby się nie zgadzają.\n";  
}
```

```
Liczba elementów w kolejce wynosi: 4  
Liczby się nie zgadzają.
```

12. Napisz metodę o nazwie printQueue, która wyświetli po kolei, od ostatniego do pierwszego, wartości wszystkich elementów znajdujących się w kolejce.

```
void printQueue() {  
    if (isEmpty()) {  
        cout << queueEmptyInfo << "\n";  
        return;  
    }  
    for (int i = last - 1; i >= first; i--) {  
        cout << arr[i] << " ";  
    }  
    cout << "\n";  
}
```

```
67 41 30 20
```

13. Napisz metodę o nazwie resetQueue, która zresetuje kolejkę, czyli przywróci ją do stanu początkowego. Następnie wyświetl całą kolejkę metodą printQueue, by udowodnić wykonanie tego podpunktu. Następnie dodaj do kolejki i usuń z niej kilka elementów.

```
void resetQueue() {  
    first = 0;  
    last = 0;  
}
```

```
mojaKolejka.resetQueue();  
mojaKolejka.printQueue();  
mojaKolejka.enqueue(55);  
mojaKolejka.enqueue(65);  
mojaKolejka.enqueue(75);  
mojaKolejka.printQueue();  
mojaKolejka.dequeue();  
mojaKolejka.printQueue();
```



```
Queue is empty.  
75 65 55  
75 65
```

Wnioski:

W trakcie ćwiczenia potwierdzono, że stos i kolejka to skuteczne sposoby na układanie danych w pamięci programu a różnica między nimi polega na kolejności obsługi elementów. Zauważono, że w stosie najpierw wyciąga się to co dodano na końcu natomiast w kolejce pierwszy dodany element jest usuwany jako pierwszy. Zaobserwowano również, że do poprawnego działania obu struktur wystarczy odpowiednio zmieniać numery indeksów tablicy co pozwala na łatwe dodawanie, usuwanie, zliczanie oraz resetowanie przechowywanych danych bez konieczności czyszczenia całej pamięci komputera.